

COVERAGE_ESCAPE_AUDIT

Coverage-escape audit — force-push guard, substitution + word-boundary class

Field	Value
Revision	1
Created	2026-09-08
Last modified	2026-09-08
Status	active

Table of contents

- [What escaped](#)
- [How it was found](#)
- [Why the standing suite missed it](#)
- [Root causes](#)
- [What was registered](#)
- [Honest gaps](#)

What escaped

Six defects in constitution/scripts/hooks/guard-forbidden-commands.sh, the PreToolUse hook that mechanically enforces §11.4.113 (absolute no-force-push, a rule that by its own text has **no** escape hatch).

Five were **false negatives** — a genuine force-push the guard ALLOWED (exit 0). Measured before the fix:

Spelling	Pre-fix
echo \$(git push --force origin main)	ALLOWED
echo `git push --force origin main`	ALLOWED
(git push --force origin main)	ALLOWED
/usr/bin/git push --force origin main	ALLOWED

Spelling	Pre-fix
x \$(y \$(git push --force z) w)	ALLOWED

One was a **false positive** — an ordinary fast-forward push REFUSED (exit 2), which §11.4.201 classes as a FAIL-bluff of equal standing:

Spelling	Pre-fix
git push gitlab main > "log_\$ (date -u +%Y%m%dT%H%M%SZ).log"	REFUSED

How it was found

Not by automation. The false positive blocked a real detached GitLab push during ordinary work; investigating that one refusal (§11.4.102) surfaced the five false negatives. Per §11.4.238 this is a **coverage escape**: discovery by live use rather than by the standing regime is itself a defect of equal standing to the bug, and closing only the bug would leave the real hole open.

Why the standing suite missed it

test_guard_forbidden_commands.sh held 32 cases and passed 32/32 against the defective guard. Two structural blind spots, both invisible from inside the suite’s own fixture style:

1. **Every** force-push fixture spelled the invocation with `git` at start-of-string or immediately after a space — the exact two positions the old anchor (`^([:space:]))` `git` recognises. No fixture ever placed a LIVE `git push` behind a non-space boundary, so the anchor’s narrowness could not be observed.
2. Every “should not block” fixture put its trigger token in an **inert** region (a quoted string or a `#` comment), which the scrubber blanks. No fixture put a `+`-leading token belonging to an **unrelated live command** in the same clause as a real push, so the `+<refspec>` heuristic’s over-match could not be observed either.

The suite was internally consistent and genuinely falsifiable — it simply tested the shapes its author had in mind. That is the ordinary way coverage escapes happen, and why §11.4.238 requires naming the missing check rather than recording “we didn’t think of it”.

Root causes

Two, both about command substitution, and independently sufficient:

1. **Boundary anchor too narrow.** `(^|[:space:])*git` requires whitespace before `git`. In `$(git ...)` the preceding char is `(;` in ``git ...`` it is a backtick; in `/usr/bin/git` it is `/`. All are genuine word boundaries; none is whitespace. Fixed by anchoring on `(^|^[[:alnum:]]_-)*git`, which excludes only the characters that would make `git` a substring of a longer identifier — so `legit push/mygit push` still correctly do not match.
2. **Substitution bodies scanned as outer-command arguments.** A substitution is a *different* command. Its tokens were pooled into the outer clause, so `date's +%Y...` satisfied the `+<refspec>` test while the outer `git push` satisfied the `push` test, and the AND fired.

Fixed by **extraction, not splitting**. Splitting the string at `$(/)` would have broken the clause splitter's load-bearing invariant — *malformed input can only MERGE clauses, never SPLIT a real force-push apart* — and would have torn `git push $(get_remote) --force` into pieces, MISSING a real force-push. Instead each body is emitted as its own command while the outer keeps an inert `@@SUB@@` placeholder, so `git push` and `--force` stay in the same outer clause.

Note the fixes **overlap**: reverting (1) alone still leaves 2 of the 5 escapes closed, because (2) hands a substitution body to the scanner as a standalone command where `git` sits at start-of-string. Defense in depth, not redundancy.

What was registered

Ten cases added to the standing suite (32 → 42), covering all five escape routes, the load-bearing “force flag after a substitution argument” case, the false positive, and a negative control proving `legit` is still not `git`.

Falsifiability proven by paired \$1.1 mutation against the **standing** suite, not a scratch harness:

- revert the boundary widening → PASS=40 FAIL=2
- neuter the substitution expander → PASS=41 FAIL=1
- restored → PASS=42 FAIL=0, mutation-residue scan clean (\$11.4.84)

Honest gaps

- The fix is proven at the **hook** layer by driving the hook directly. It is additionally confirmed at the live layer for exactly one command — the real

push that was wrongly refused now passes. Other spellings are covered by fixtures, not by live invocation.

- The escape-route list is the set found by one adversarial pass. §11.4.118 applies: this reduces the unknown-unknown surface, it does not prove the guard now has zero remaining escapes. An independent adversarial review was dispatched specifically to hunt for spellings this pass did not imagine.
- Whether the five escapes were ever actually *used* to land a force-push is NOT established here. `git reflog/upstream` history were not audited for evidence of a past force-push through these routes. UNKNOWN, not “no”.